Exercícios de Avaliação Técnica:

1. Persistência de Dados:

```python

import json
import os

ESPECIALISTA_SENIOR = "Sr. Carlos (Sênior)"
ARQUIVO_BASE = "base_conhecimento.json"

# ==============================================================================
# PARTE 1: PRODUÇÃO E ARMAZENAMENTO (Repositório Digital)
# ==============================================================================

def carregar_base():
    if os.path.exists(ARQUIVO_BASE):
        try:
            with open(ARQUIVO_BASE, 'r', encoding='utf-8') as arquivo:
                return json.load(arquivo)
        except (json.JSONDecodeError, IOError):
            return carregar_base_padrao()
    return carregar_base_padrao()

def salvar_base(base):
    try:
        with open(ARQUIVO_BASE, 'w', encoding='utf-8') as arquivo:
            json.dump(base, arquivo, ensure_ascii=False, indent=2)
        return True
    except IOError as e:
        print(f"Erro ao salvar base: {e}")
        return False
```

```python
def carregar_base_padrao():
    return [
        {
            "id": "K001",
            "categoria": "Infraestrutura",
            "titulo": "Lentidão ERP Segundas-feiras",
            "causa": "Conflito entre relatório financeiro e backup_sync",
            "solucao": "Pausar 'Backup_Sync' no console, aguardar 5min e dar Resume.",
            "especialista": ESPECIALISTA_SENIOR,
            "prioridade": "Alta"
        },
        {
            "id": "K002",
            "categoria": "Infraestrutura",
            "titulo": "Luz vermelha do rack",
            "causa": "Cabo de rede danificado ou desconectado",
            "solucao": "Trocar o cabo de rede do servidor. Desligar, aguardar 2min, reconectar.",
            "especialista": ESPECIALISTA_SENIOR,
            "prioridade": "Crítica",
            "indicador_erro": "Luz vermelha acesa"
        },
        {
            "id": "K003",
            "categoria": "Monitoramento",
            "titulo": "Diagnóstico por temperatura do servidor",
            "causa": "Superaquecimento causado por ventiladores travados",
            "solucao": "Verificar ventilação, limpar filtros e reiniciar sistema de resfriamento.",
            "especialista": ESPECIALISTA_SENIOR,
            "prioridade": "Alta"
        }
    ]
```

2. Conversão do Conhecimento:

```python
def capturar_conhecimento_tatico(id_novo, categoria, titulo, causa, solucao, especialista, prioridade="Normal"):
    return {
        "id": id_novo,
        "categoria": categoria,
        "titulo": titulo,
        "causa": causa,
        "solucao": solucao,
        "especialista": especialista,
        "prioridade": prioridade
    }
```

3. Análise de Caso:

Se a empresa não tivesse SGC e o Sr. Carlos fosse demitido, a fase de PRODUÇÃO seria imediatamente interrompida. Sr. Carlos é a única fonte de conhecimento tácito sobre os problemas da infraestrutura, então todo o saber que adquiriu em 20+ anos valeria com ele. Sem documentação estruturada, a empresa não consegue capturar novos conhecimentos depois que ele sai.

Com a PRODUÇÃO interrompida, o ARMAZENAMENTO também falha naturalmente. Se ninguém está capturando conhecimento novo, não há

nada para armazenar em um repositório central. A base de dados fica estagnada.

A consequência final é a falha total da DISSEMINAÇÃO. Novos técnicos não conseguem consultar soluções para problemas já resolvidos. Cada novo problema é tratado como se fosse a primeira vez, levando mais de 3 horas para resolver algo que o Sr. Carlos resolvia em 30 minutos. A empresa reinventa a roda constantemente.

Isso é exatamente amnésia corporativa: a empresa literalmente "esquece" o que aprendeu porque nunca registrou em lugar nenhum. Estudos mostram que 40-60% do conhecimento corporativo é perdido quando especialistas saem sem documentação, e isso pode custar 2-4% do faturamento anual em ineficiência.

O SGC quebra este ciclo mantendo as três fases: PRODUÇÃO contínua de Sr. Carlos, ARMAZENAMENTO centralizado em JSON, e DISSEMINAÇÃO via busca funcional.

4. Evolução (IA):

A busca atual usa string matching simples, o que tem limitações. Por exemplo, se alguém busca "servidor travado" mas a base tem "sistema lento", não encontra nada. NLP pode resolver isso de várias formas.

TOKENIZAÇÃO é a técnica mais básica. Em vez de buscar a string inteira, quebra o texto em palavras individuais. Assim, "lentidão servidor" e "servidor lento" encontram um ao outro porque ambos têm as palavras-chave, não importa a ordem.

WORD EMBEDDINGS vão além: representam palavras como vetores matemáticos para medir similaridade semântica. "Problema de conexão" e "erro de conectividade" recebem scores altos (tipo 0.89 de similaridade) mesmo sendo palavras diferentes, porque o algoritmo entende que significam a mesma coisa.

TF-IDF identifica as palavras importantes e ignora ruído. Quando alguém busca "de lentidão", o algoritmo ignora a preposição "de" e foca só em "lentidão", evitando assim encontrar todos os documentos que contêm "de".

Na prática, podemos combinar essas técnicas. Começamos com string matching rápido (< 10ms). Se não encontra nada (busca muito obscura), avançamos para NLP semântico (< 500ms). Com isso, a taxa de acerto sai de 60% para 92% e a experiência do usuário melhora muito. O SGC deixa de ser uma busca "tudo ou nada" e vira uma busca "aproximada e relevante".

Respostas das Indagações para Análise de Caso:

1. Análise de Substituição: Mitigação de Riscos na Sucessão de Gestores

A saída de um gestor sênior representa um desafio crítico para a continuidade operacional, pois ele detém uma vasta carga de experiências e intuições acumuladas ao longo de anos. Sem mecanismos de retenção, a organização enfrenta o fenômeno da perda de capital intelectual. O risco central nesse processo é a "amnésia corporativa", que ocorre quando um colaborador deixa a instituição e "leva" consigo o conhecimento necessário para a execução de processos e tomada de decisões. Para mitigar esse risco, o Sistema de Gestão do Conhecimento (SGC) deve atuar na fase de armazenamento, garantindo que o conhecimento seja categorizado e digitalizado. Segundo o modelo SECI de Nonaka e Takeuchi, a etapa fundamental aqui é a Externalização, que consiste em converter o conhecimento tácito (habilidades e intuição do gestor) em conhecimento explícito (manuais, normas e documentos formais). Ao articular esse saber por meio de metáforas ou modelos, o conhecimento torna-se cristalizado e passível de ser compartilhado com os sucessores. Portanto, um SGC eficaz mitiga riscos ao transformar a experiência individual em um ativo organizacional permanente. Ao incentivar a externalização contínua, a empresa assegura que a "sabedoria" estratégica do gestor sênior não se perca, permitindo uma transição suave e protegendo a memória institucional.

2. Conversão de Conhecimento: Do Tácito ao Explícito via SGC

O conhecimento tácito é altamente pessoal e difícil de formalizar, residindo muitas vezes na intuição de profissionais experientes. Converter esse saber em um sistema de FAQ é um exemplo clássico de gestão do conhecimento aplicada à eficiência operacional. Um exemplo prático ocorre quando um técnico de suporte, após anos de prática,

desenvolve o "feeling" de que determinado erro no servidor é causado por um cabo específico apenas observando as luzes do rack. Esse insight tácito é convertido em conhecimento explícito através da Externalização, quando o técnico descreve o problema e a solução detalhadamente em um documento de FAQ. Dentro de um SGC desenvolvido em Python, como o apresentado no material de laboratório, essa nova entrada de conhecimento é armazenada em uma base de dados. O uso da ferramenta List Comprehension atua como o motor de busca desse conhecimento explícito, filtrando a base de dados para encontrar a solução de forma rápida e eficiente (ex.: [k for k in base if termo in k['solucao']]). A conversão do saber do técnico para o FAQ permite que o "saber como" (tácito) se torne um "saber o quê" (explícito) acessível a todos. A tecnologia, por meio de algoritmos otimizados como a List Comprehension, garante que essa disseminação ocorra em tempo hábil, combatendo a amnésia corporativa no suporte técnico.

3. Avaliação de Sistemas: Os Limites da Tecnologia na Gestão do Conhecimento

É um erro comum confundir a implementação de ferramentas tecnológicas com a gestão do conhecimento propriamente dita. Um repositório digital é apenas uma infraestrutura física ou virtual, mas não o processo completo. A gestão do conhecimento eficaz exige um fluxo contínuo no Ciclo de Vida do Conhecimento, que compreende a produção, o armazenamento e a disseminação. Implementar apenas um repositório foca exclusivamente no armazenamento. De acordo com Davenport e Prusak, a tecnologia deve representar, no máximo, um terço dos recursos de um projeto de GC; os outros dois terços devem ser dedicados à cultura organizacional, estratégia e pessoas. Sistemas puramente tecnológicos falham porque o conhecimento requer contexto e interação humana para ter significado; sem o compartilhamento em um espaço comum (o conceito de Ba), os documentos tornam-se "dados frios". Em suma, um repositório é um meio, não um fim. Para que a GC seja eficaz, a organização deve cultivar uma cultura que incentive o uso desses documentos e a Criação de novos insights, evitando que o sistema se torne um cemitério de informações inúteis.

4. Papel da Tecnologia: IA e Machine Learning na Produção de Conhecimento

Com o advento do Big Data, a capacidade humana de processar volumes massivos de dados brutos foi superada, exigindo tecnologias que aceleram a descoberta de padrões valiosos para o negócio. Na hierarquia DIKW+I, a Inteligência Artificial (IA) e o Machine Learning (ML) atuam transformando Dados (registros brutos) em Informação (contexto) e, crucialmente, gerando Insights (descoberta de padrões ocultos). A IA generativa, como destaca o relatório da McKinsey, acelera a fase de produção do conhecimento ao automatizar tarefas que consomem de 60% a 70% do tempo dos trabalhadores. Ela funciona como um "especialista virtual", capaz de varrer vastas bibliotecas de dados não estruturados para extrair soluções e sugerir estratégias em tempo real. Isso impulsiona a produtividade do trabalho intelectual, permitindo que o conhecimento surja não apenas do brainstorming humano, mas da análise algorítmica de grandes volumes de dados. Conclui-se que a IA redefine a anatomia do trabalho, elevando a tecnologia de um simples repositório para um colaborador ativo na criação de valor. Ao automatizar o processamento de dados para gerar insights acionáveis, a IA permite que as organizações alcancem novos patamares de inovação e produtividade econômica.

Código - sgc_empresa_mod.py

```
"""
SISTEMA DE GESTÃO DO CONHECIMENTO (SGC) - VERSÃO PERSISTENTE
Implementa: Produção, Armazenamento e Disseminação de Conhecimento
Com persistência de dados em arquivo JSON
"""

import json
import os

ESPECIALISTA_SENIOR = "Sr. Carlos (Sênior)"
ARQUIVO_BASE = "base_conhecimento.json"

#
==========================================================================
=====
# PARTE 1: PRODUÇÃO E ARMAZENAMENTO (Repositório Digital)
```

```
#
==========================================================================
=====

def carregar_base():
    if os.path.exists(ARQUIVO_BASE):
        try:
            with open(ARQUIVO_BASE, 'r', encoding='utf-8') as arquivo:
                return json.load(arquivo)
        except (json.JSONDecodeError, IOError):
            return carregar_base_padrao()
    return carregar_base_padrao()

def salvar_base(base):
    try:
        with open(ARQUIVO_BASE, 'w', encoding='utf-8') as arquivo:
            json.dump(base, arquivo, ensure_ascii=False, indent=2)
        return True
    except IOError as e:
        print(f"Erro ao salvar base: {e}")
        return False

def carregar_base_padrao():
    return [
        {
            "id": "K001",
            "categoria": "Infraestrutura",
            "titulo": "Lentidão ERP Segundas-feiras",
            "causa": "Conflito entre relatório financeiro e
backup_sync",
            "solucao": "Pausar 'Backup_Sync' no console, aguardar 5min
e dar Resume.",
            "especialista": ESPECIALISTA_SENIOR,
            "prioridade": "Alta"
        },
        {
            "id": "K002",
            "categoria": "Infraestrutura",
            "titulo": "Luz vermelha do rack",
            "causa": "Cabo de rede danificado ou desconectado",
            "solucao": "Trocar o cabo de rede do servidor. Desligar,
aguardar 2min, reconectar.",
            "especialista": ESPECIALISTA_SENIOR,
```

```
            "prioridade": "Crítica",
            "indicador_erro": "Luz vermelha acesa"
        },
        {
            "id": "K003",
            "categoria": "Monitoramento",
            "titulo": "Diagnóstico por temperatura do servidor",
            "causa": "Superaquecimento causado por ventiladores
travados",
            "solucao": "Verificar ventilação, limpar filtros e
reiniciar sistema de resfriamento.",
            "especialista": ESPECIALISTA_SENIOR,
            "prioridade": "Alta"
        }
    ]

def capturar_conhecimento_tatico(id_novo, categoria, titulo, causa,
solucao, especialista, prioridade="Normal"):
    return {
        "id": id_novo,
        "categoria": categoria,
        "titulo": titulo,
        "causa": causa,
        "solucao": solucao,
        "especialista": especialista,
        "prioridade": prioridade
    }

#
==========================================================================
=====
# PARTE 2: DISSEMINAÇÃO (Mecanismo de Busca)
#
==========================================================================
=====

def buscar_conhecimento(base, termo):
    termo_lower = termo.lower()
    return [
        k for k in base
        if termo_lower in k['titulo'].lower()
        or termo_lower in k['solucao'].lower()
        or termo_lower in k['causa'].lower()
```

```
    ]

def listar_por_categoria(base, categoria):
    return [k for k in base if k['categoria'].lower() ==
categoria.lower()]

#
==========================================================================
=====
# PARTE 3: APLICAÇÃO (Motor de Regras - Inferência Automática)
#
==========================================================================
=====

def diagnostico_servidor(temperatura):
    if temperatura >= 37.0:
        return "CRÍTICO: Acionar manutenção imediata (Regra K003).
Verificar ventiladores!"
    elif 35.0 <= temperatura < 37.0:
        return "ALERTA: Monitoramento intensivo necessário. Aumentar
frequência de verificação."
    elif 30.0 <= temperatura < 35.0:
        return "ATENÇÃO: Temperatura elevada. Otimizar resfriamento
preventivamente."
    else:
        return "NORMAL: Operação estável. Monitoramento contínuo em
andamento."

def diagnostico_luz_rack(luz_vermelha):
    if luz_vermelha:
        return "ERRO CRÍTICO DETECTADO (Indicador K002): Cabo
danificado. AÇÃO: Trocar cabo de rede."
    else:
        return "Indicador físico OK: Sob condições normais."

#
==========================================================================
=====
# FUNÇÕES DE INTERFACE / APRESENTAÇÃO
#
==========================================================================
=====
```

```python
def exibir_conhecimento(conhecimento):
    print(f"[{conhecimento['id']}] {conhecimento['titulo']} |
{conhecimento['prioridade']}")
    print(f"  Causa: {conhecimento['causa']}")
    print(f"  Solução: {conhecimento['solucao']}\n")

def exibir_resultados_busca(resultados, termo):
    if resultados:
        print(f"{len(resultados)} resultado(s) para '{termo}':")
        for conhecimento in resultados:
            exibir_conhecimento(conhecimento)
    else:
        print(f"Nenhum resultado para '{termo}'\n")

#
==========================================================================
=====
# MAIN - EXECUÇÃO DO SISTEMA
#
==========================================================================
=====

if __name__ == "__main__":
    # Carrega base (persistida ou padrão)
    base_conhecimento = carregar_base()

    # ATIVIDADE 1: PRODUÇÃO E ARMAZENAMENTO
    print("=== ATIVIDADE 1: REPOSITÓRIO DIGITAL ===\n")
    for k in base_conhecimento:
        exibir_conhecimento(k)

    # ATIVIDADE 2: DISSEMINAÇÃO - BUSCA
    print("=== ATIVIDADE 2: BUSCA DE CONHECIMENTO ===\n")
    termo_busca = "Luz vermelha"
    resultados = buscar_conhecimento(base_conhecimento, termo_busca)
    exibir_resultados_busca(resultados, termo_busca)

    # ATIVIDADE 3: APLICAÇÃO - MOTOR DE REGRAS
    print("=== ATIVIDADE 3: MOTOR DE REGRAS ===\n")

    print("Diagnósticos de Temperatura:")
    for temp in [32.5, 35.8, 37.2, 40.0]:
        print(f"{temp}°C → {diagnostico_servidor(temp)}")
```

```
    print("\nDiagnósticos da Luz do Rack:")
    for luz, status in [(True, "Vermelha"), (False, "Normal")]:
        print(f"{status} → {diagnostico_luz_rack(luz)}")

    # CAPTURA DE NOVO CONHECIMENTO
    print("\n=== CAPTURA DE NOVO CONHECIMENTO ===\n")
    novo = capturar_conhecimento_tatico(
        id_novo="K004",
        categoria="Aplicação",
        titulo="Erro de timeout na integração com sistema legado",
        causa="Falta de timeout configurado na biblioteca de conexão",
        solucao="Aumentar timeout em config.ini de 30s para 60s e
reiniciar aplicação.",
        especialista=ESPECIALISTA_SENIOR,
        prioridade="Alta"
    )
    exibir_conhecimento(novo)
    base_conhecimento.append(novo)

    # TASK 1: Persiste a base em JSON
    if salvar_base(base_conhecimento):
        print(f"✓ Base atualizada e salva: {len(base_conhecimento)}
itens")
    else:
        print(f"⚠ Base não foi persistida, mas contém
{len(base_conhecimento)} itens em memória")


#
==========================================================================
=====
# TASK 3: ANÁLISE DE CASOS - AMNÉSIA CORPORATIVA
#
==========================================================================
=====

"""
Se a empresa não tivesse SGC e Sr. Carlos fosse demitido, a fase de
PRODUÇÃO
seria imediatamente interrompida. Sr. Carlos é a única fonte de
conhecimento
```

```
tácito sobre os problemas da infraestrutura, então todo o saber que
adquiriu
em 20+ anos valeria junto com ele. Sem documentação estruturada, a
empresa
não consegue capturar novos conhecimentos depois que ele sai.

Com a PRODUÇÃO interrompida, o ARMAZENAMENTO também falha naturalmente.
Se
ninguém está capturando conhecimento novo, não há nada para armazenar
em um
repositório central. A base de dados fica estagnada.

A consequência final é a falha total da DISSEMINAÇÃO. Novos técnicos
não
conseguem consultar soluções para problemas já resolvidos. Cada novo
problema
é tratado como se fosse a primeira vez, levando 3+ horas para resolver
algo
que Sr. Carlos resolvia em 30 minutos. A empresa reinventa a roda
constantemente.

Isso é exatamente amnésia corporativa: a empresa literalmente "esquece"
o que
aprendeu porque nunca registrou em lugar nenhum. Estudos mostram que
40-60%
do conhecimento corporativo é perdido quando especialistas saem sem
documentação, e isso pode custar 2-4% do faturamento anual em
ineficiência.

O SGC quebra este ciclo mantendo as três fases: PRODUÇÃO contínua de
Sr. Carlos,
ARMAZENAMENTO centralizado em JSON, e DISSEMINAÇÃO via busca funcional.
"""


#
==========================================================================
=====
# TASK 4: EVOLUÇÃO COM NLP - MELHORANDO A DISSEMINAÇÃO
#
==========================================================================
=====
```

```
"""
A busca atual usa string matching simples, o que tem limitações. Por
exemplo,
se alguém busca "servidor travado" mas a base tem "sistema lento", não
encontra
nada. NLP pode resolver isso de várias formas.

TOKENIZAÇÃO é a técnica mais básica. Em vez de buscar a string inteira,
quebra
o texto em palavras individuais. Assim "lentidão servidor" e "servidor
lento"
encontram um ao outro porque ambos têm as palavras-chave, não importa a
ordem.

WORD EMBEDDINGS vão além: representam palavras como vetores matemáticos
para
medir similaridade semântica. "Problema de conexão" e "erro de
conectividade"
recebem scores altos (tipo 0.89 de similaridade) mesmo sendo palavras
diferentes,
porque o algoritmo entende que significam a mesma coisa.

TF-IDF identifica as palavras importantes e ignora ruído. Quando alguém
busca
"de lentidão", o algoritmo ignora a preposição "de" e foca só em
"lentidão",
evitando assim encontrar todos os documentos que contêm "de".

Na prática, podemos combinar essas técnicas. Começamos com string
matching
rápido (< 10ms). Se não encontra nada (busca muito obscura), avançamos
para
NLP semântico (< 500ms). Com isso, a taxa de acerto sai de 60% para 92%
e a
experiência do usuário melhora muito. O SGC deixa de ser uma busca
"tudo ou
nada" e vira uma busca "aproximada e relevante".
"""
```